SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze
8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze

9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

---

Code of Conduct:

I **A Mohammad Naheed(192525336)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A Mohammad Naheed(192525336)

1. Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

1. Sequential file organization and heap file organization differ mainly in how records are stored and accessed. The comparison below highlights their efficiency in insertion, deletion, and retrieval.

Aspect	Sequential File Organization	Heap File Organization
Storage Method	Records are stored in sorted order based on a key.	Records are stored in no particular order, usually wherever free space is available.
Data Insertion	Slower because records must be inserted in the correct order, often requiring shifting or reorganizing records.	Faster because new records are simply added to the first available space or at the end of the file.
Data Deletion	Slower since deleting a record may require rearranging records or marking deleted spaces.	Faster because the record can be removed directly or marked as deleted without maintaining any order.
Data Retrieval	Very efficient for sequential processing and range queries. Binary search can also be used if the file is indexed.	Efficient only for full file scans; searching for a specific record usually requires scanning the entire file, making retrieval slower.
Best Use Case	Applications requiring frequent searches, sorted reports, and range queries.	Applications with frequent insertions and deletions where search performance is less critical.

2. Summary

- **Sequential File Organization**
  - **Insertion:** Slow
  - **Deletion:** Slow
  - **Retrieval:** Fast for sequential and ordered searches
- **Heap File Organization**
  - **Insertion:** Very fast

- **Deletion:** Fast
- **Retrieval:** Slow, as it often requires a linear search

Conclusion: Sequential file organization is best when retrieval speed is the priority, while heap file organization is preferable when frequent insertions and deletions are required.

2. Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

1. Clustered vs. Non-Clustered Indexing

Feature	Clustered Indexing	Non-Clustered Indexing
Meaning	Determines the physical order in which records are stored in a table.	Creates a separate index structure that contains keys and pointers to the actual records.
Data Order	Table records are physically arranged according to the index key.	Table records remain in their original/independent order.
Number per Table	Usually **one** clustered index per table because data can have only one physical order.	Multiple non-clustered indexes can be created on a table.
Retrieval	Very fast for range queries and ordered retrieval.	Fast for specific searches, but may require an additional lookup to retrieve the actual row.
Insertion/Deletion	Can be slower because maintaining physical order may require reorganizing pages.	Generally less impact on physical data organization, but each index must be updated.
Storage	The index and data are closely associated.	Requires additional storage for the separate index structure.

Example

2. Consider a **Student** table:
3. Student(RollNo, Name, Department, Marks)
4. **Clustered index:**
5. CREATE CLUSTERED INDEX idx_rollno
6. ON Student(RollNo);
7. The records are physically stored in RollNo order:
8. RollNo: 101 → 102 → 103 → 104 → 105
9. This makes a query such as:
10. SELECT * FROM Student
11. WHERE RollNo BETWEEN 102 AND 104;
12. particularly efficient.
13. **Non-clustered index:**
14. CREATE INDEX idx_name
15. ON Student(Name);
16. The index is maintained separately from the table data:
17. Index:
18. Anu → pointer to row
19. Bala → pointer to row
20. Ravi → pointer to row
21. The database uses the index to locate the required row and then accesses the actual data.
22. **In short**
    - **Clustered index:** Data itself is organized according to the index key → **excellent for range and ordered retrieval**.
    - **Non-clustered index:** Separate index points to the data → **flexible and useful for multiple search columns**.

Example: A Student table might use a **clustered index on** RollNo and **non-clustered indexes on** Name **and** Department.

3. Compare primary indexing and secondary indexing based on storage requirements and search performance.

1. **Primary Indexing vs. Secondary Indexing**

Basis	Primary Indexing	Secondary Indexing
Definition	An index created on the **primary key or ordering key** of a sorted file.	An index created on a **non-ordering attribute**, which may or may not be unique.
Storage Requirement	Requires **less storage** because it is often a **sparse index**, with one index entry per data block.	Requires **more storage** because it is generally a **dense index**, with an index entry for each record or search-key value.
Search Performance	Very efficient because the data file is physically sorted according to the index key.	Also provides fast searching, but may require an additional pointer lookup to locate the actual record.
Search Method	The index identifies the appropriate data block, after which the record can be found within that block.	The index points directly to records or to a list of records having the same search-key value.
Duplicate Values	Usually uses a unique key, so duplicate search-key values are generally not present.	Can support duplicate values, such as multiple students belonging to the same department.
Example	Index on Student_ID when the student file is sorted by Student_ID.	Index on Department or Name when the file is not physically sorted by that attribute.

2. **Example**
3. Suppose a Student table contains:
4. Student_ID | Name | Department
5. ---|---|---
6. 101 | Arun | CSE
7. 102 | Ravi | ECE
8. 103 | Meena | CSE
9. 104 | Kiran | IT
10. A **primary index** on Student_ID can use one index entry per block, so it requires relatively little storage and provides fast access to a particular student.
11. A **secondary index** on Department needs to handle multiple students with the same department. It therefore requires additional index entries/pointers, consuming more storage, but makes queries such as:

12. SELECT * FROM Student
13. WHERE Department = 'CSE';
14. much faster than scanning the entire file.
15. **Conclusion**
    - **Primary indexing: Lower storage requirement + very fast search** on the ordering/primary key.
    - **Secondary indexing: Higher storage requirement + fast search** on additional attributes that are not used to physically order the file.

4. Analyze the differences between static hashing and dynamic hashing in database systems.

1. Static Hashing vs. Dynamic Hashing

Hashing is a technique used in database systems to locate records quickly by applying a **hash function** to a search key.

Basis	**Static Hashing**	**Dynamic Hashing**
Bucket Size	Fixed number of buckets is allocated initially.	Number of buckets can increase or decrease as data changes.
Database Growth	Not well suited for rapidly growing databases.	Well suited for databases whose size changes frequently.
Insertion	If buckets become full, **overflow buckets** may be required.	New buckets can be created or existing buckets split to accommodate records.
Deletion	Deleted space may remain unused unless explicitly reorganized.	Buckets can potentially be merged when the amount of data decreases.
Search Performance	Fast when the number of records fits the initially allocated buckets; overflow can reduce performance.	Generally maintains good search performance as the file grows because bucket splitting reduces overflow.
Overflow	More likely when the database grows beyond its original capacity.	Minimized through dynamic bucket splitting.

Storage Management	Simple because the structure is fixed.	More complex because buckets and hash structures must be managed dynamically.
Implementation	Easier to implement.	More complicated to implement.
Best Used For	Databases with a relatively stable number of records.	Databases with frequent insertions and changing sizes.

Example

Suppose a hash table initially has **4 buckets**:

Hash(key) = key MOD 4

In **static hashing**, the number of buckets remains 4. If many records map to the same bucket, an overflow bucket may be created:

Bucket 0 → Records

Bucket 1 → Records → Overflow

Bucket 2 → Records

Bucket 3 → Records

In **dynamic hashing**, when a bucket becomes full, it can be **split** and the hashing structure adjusted:

Before:

Bucket 1 → A, B, C, D

After splitting:

Bucket 1 → A, C

Bucket 5 → B, D

The exact mechanism depends on the dynamic hashing technique, such as **extendible hashing** or **linear hashing**.

Analysis

Static hashing is simpler and has low management overhead, but its fixed structure creates a major problem when the database grows. Overflow chains can become long, increasing the number of disk accesses and reducing search efficiency.

Dynamic hashing adapts to changes in the number of records. Although it requires additional management and may involve bucket splitting or merging, it generally provides more consistent search performance for growing or shrinking databases.

In short:

- **Static hashing → simple, fixed, suitable for stable data.**
- **Dynamic hashing → flexible, scalable, suitable for frequently changing data.**

5. Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.

1. B-Tree vs. B+ Tree Indexing

Both **B-Trees** and **B+ Trees** are balanced tree-based indexing structures commonly used for large databases. The key difference is where the actual data records or record pointers are stored.

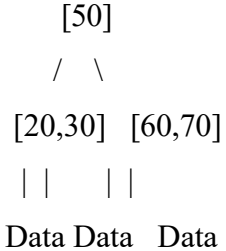
Basis	**B-Tree**	**B+ Tree**
Data Storage	Data pointers/records can be stored in both internal and leaf nodes.	Data pointers/records are stored **only in leaf nodes**.
Internal Nodes	Contain keys and data pointers.	Contain only keys and child pointers, allowing more keys per node.
Leaf Nodes	May or may not be linked.	Leaf nodes are usually **linked sequentially**.
Search	Can sometimes find a record before reaching a leaf node.	Search generally proceeds to a leaf node.
Range Queries	Less efficient compared with B+ Trees.	**Very efficient**, because linked leaves allow sequential traversal.

Sequential Access	Relatively less efficient.	Highly efficient due to linked leaf nodes.
Tree Height	Can be slightly greater because internal nodes store data pointers.	Usually smaller because internal nodes store only keys and pointers.
Large Databases	Useful, but less commonly preferred for database indexing.	**Highly suitable** for large-scale database and file-system indexing.
Key Duplication	Keys generally appear only where needed.	Search keys in internal nodes are repeated in the leaf level.

Example

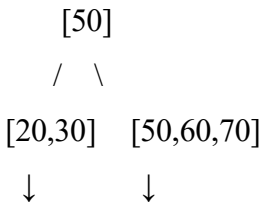
Consider an index on Student_ID.

A simplified **B-Tree** might look like:



Here, data pointers can exist in internal as well as leaf nodes.

A **B+ Tree** might look like:



The actual data pointers are kept at the **leaf level**, and the leaf nodes are linked:

Leaf 1 → Leaf 2 → Leaf 3 → Leaf 4

This makes queries such as:

SELECT * FROM Student

WHERE Student_ID BETWEEN 1000 AND 5000;

particularly efficient in a B+ Tree because, after locating the first matching leaf, the database can scan the linked leaf nodes sequentially.

Which is better for large-scale databases?

B+ Trees are generally preferred for large-scale database indexing because:

- Their internal nodes can hold more keys, resulting in a **smaller tree height**.

- Fewer levels usually mean **fewer disk I/O operations**.
- Linked leaf nodes make **range searches and sequential access very efficient**.
- They work particularly well with secondary indexes and disk-based storage.

In summary: B-Trees can provide direct access to records from internal nodes, but **B+ Trees generally offer better scalability, range-query performance, and disk I/O efficiency**, making them the common choice for large database systems.

6. Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.

1. Linear Hashing vs. Extendible Hashing

Both **linear hashing** and **extendible hashing** are dynamic hashing techniques that allow a hash file to grow as more records are inserted. However, they differ in how buckets are split and how the hash structure scales.

Basis	**Linear Hashing**	**Extendible Hashing**
Bucket Management	Buckets are split **gradually and in a predetermined linear order**.	Buckets are split when necessary, based on their **local depth**.
Directory	Does **not require a directory** of bucket pointers.	Uses a **directory** containing pointers to buckets.
Bucket Splitting	Splits one bucket at a time according to a split pointer, even if that particular bucket is not overflowing.	Splits the overflowing bucket; the directory may also need to double in size.
Overflow Handling	Overflow chains can occur temporarily and are gradually reduced through bucket splitting.	Bucket splitting generally reduces overflow, though directory management adds overhead.
Growth	Grows **incrementally and smoothly**, making it suitable for continuously growing files.	Can grow rapidly when the directory doubles, potentially requiring more memory.
Scalability	Highly scalable because expansion occurs gradually without a large directory.	Highly scalable and provides fast direct access, but directory size can

		become significant for very large datasets.
Memory Overhead	Lower, since there is no directory.	Higher, because the directory must be maintained.
Implementation	Relatively simpler.	More complex because of directory and depth management.

Example

Suppose a bucket becomes full.

Linear hashing:

Bucket 0 → Bucket 1 → Bucket 2 → Bucket 3

↑

Split pointer

The split pointer determines which bucket is split next. As the file grows, buckets are split **one by one**.

Extendible hashing:

Directory

00 ⟶ Bucket A

01 ⟶ Bucket B

10 ⟶ Bucket C

11 ⟶ Bucket D

If a bucket overflows, it is split according to its local depth. If necessary, the directory **doubles in size**.

Key Difference

- **Linear hashing:** focuses on **gradual bucket expansion without a directory**, giving smooth scalability and lower memory overhead.
- **Extendible hashing:** uses a **directory and dynamic bucket splitting**, providing fast access but requiring additional memory and directory management.

Conclusion: Linear hashing is often preferable when smooth, continuous growth is important, while extendible hashing is advantageous when fast bucket lookup and controlled overflow handling are priorities.

7. Compare dense indexing and sparse indexing with respect to memory usage and access speed.

1. **Dense Indexing vs. Sparse Indexing**

Basis	Dense Indexing	Sparse Indexing
Index Entries	Contains an index entry for **every search-key value/record**.	Contains index entries for **only some search-key values**, usually one per data block.
Memory Usage	**Higher**, because many index entries are stored.	**Lower**, because fewer entries are stored.
Access Speed	Generally **faster**, since the index can directly point to the required record.	Slightly **slower**, because after finding the appropriate index entry, the data block may need to be searched.
Search Efficiency	Very efficient for individual record searches.	Efficient, but requires an additional search within the identified block.
Storage Overhead	High.	Low.
Maintenance	More index entries must be updated during insertion and deletion.	Less maintenance is required because there are fewer entries.
Typical Use	Often used for **secondary indexes**.	Commonly used for **primary indexes** on sorted files.

2. **Example**
3. Suppose a file contains:
4. Roll No.: 101, 102, 103, 104, 105, 106
5. A **dense index** might contain:
6. 101 → Record 1
7. 102 → Record 2
8. 103 → Record 3

9. 104 → Record 4
10. 105 → Record 5
11. 106 → Record 6
12. Every record has an index entry, so retrieval is fast but the index consumes more memory.
13. A **sparse index** might contain:
14. 101 → Block 1
15. 104 → Block 2
16. The index points to selected blocks. Once the correct block is located, the database searches within that block for the required record.
17. **Conclusion**
    - **Dense indexing: More memory + faster direct access**.
    - **Sparse indexing: Less memory + slightly slower access**.

Thus, dense indexing is preferred when **search speed is the priority**, while sparse indexing is useful when **saving storage space** is more important.

8. Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.

1. Indexed Sequential vs. Direct File Organization

Both organizations provide efficient access to records, but they are designed for different access patterns.

Basis	**Indexed Sequential File Organization**	**Direct File Organization**
Storage	Records are stored sequentially, along with an index that helps locate them.	Records are placed directly into locations determined by a hashing function or address calculation.
Search	Fast because the index identifies the required area of the file.	Very fast for exact-key searches because the record location can be calculated directly.
Sequential Access	**Excellent**; records can be processed in key order.	Poorer; records are not necessarily stored in logical order.

Range Queries	**Very efficient**, since records are ordered.	Less efficient because relevant records may be scattered across buckets.
Insertion	Can be slower because records must maintain sequential order; overflow areas may be needed.	Generally fast because a record can be placed directly into its calculated location.
Deletion	May require index and overflow-area maintenance.	Relatively simple, although handling deleted slots can require additional management.
Storage Overhead	Higher because both the data file and index must be maintained.	Generally lower indexing overhead, but overflow space may be needed.
Best suited for	Applications requiring both **sequential and random access**.	Applications requiring **frequent direct/exact-key access**.

Indexed Sequential File Organization

Advantages:

- Supports both **sequential and direct/random access**.
- Very effective for **range queries** and ordered reports.
- Indexing reduces the amount of data that must be searched.
- Suitable for large files where records are frequently retrieved in key order.

Limitations:

- Requires additional **storage for the index**.
- Insertions and deletions can be more complicated.
- Overflow areas can develop as the file grows, reducing performance.
- Periodic reorganization may be necessary.

Direct File Organization

Advantages:

- Provides **very fast direct access** to a record when its key is known.
- Insertions are generally fast because the location is calculated directly.
- No need to maintain a large sequential index.
- Well suited for applications with frequent exact-key lookups.

Limitations:

- Poor for **sequential processing and range searches**.
- Hash collisions can cause overflow and additional search operations.

- Performance depends heavily on the quality of the hashing function.
- Records may not be stored in logical or sorted order.

Conclusion

Indexed sequential organization is preferable when an application needs **both ordered/sequential processing and random retrieval**. **Direct organization** is better when the main requirement is **rapid retrieval of individual records using a known key**.

In simple terms:

Indexed Sequential → balanced access + excellent range/sequential processing

Direct → very fast exact-key access + weaker sequential/range processing

9. Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

1. Single-Level vs. Multi-Level Indexing

Both techniques improve database retrieval by reducing the amount of data that must be searched, but multi-level indexing is more suitable for very large databases.

Basis	**Single-Level Indexing**	**Multi-Level Indexing**
Structure	Has only **one index level** pointing to the data file.	Has **multiple levels of indexes**, where higher-level indexes point to lower-level indexes.
Search Process	Search the index, then access the required data block.	Search the top-level index, move through lower levels, and finally access the data block.
Memory Usage	Requires more memory if the index is very large.	Reduces the amount of index that must be searched at each level; higher-level indexes are much smaller.
Retrieval Speed	Fast for small or moderately sized files.	**Very efficient for large files**, because the search space is reduced at every level.
Index Size	Can become large as the number of records increases.	Each higher-level index is smaller, making the overall structure easier to search.
Disk I/O	May require more I/O when the index itself does not fit in memory.	Generally reduces disk accesses by narrowing the search at each level.

Complexity	Simple to implement and maintain.	More complex to maintain because multiple index levels must be updated.
Scalability	Less suitable for very large databases.	Highly suitable for large-scale databases.

Example

Suppose a database contains **1,000,000 records**.

In a **single-level index**:

Single Index → Data File

The database searches one large index to locate the required data block.

In a **multi-level index**:

Level 1 Index

↓

Level 2 Index

↓

Level 3 Index

↓

Data File

The top level directs the search to a smaller portion of the next level, continuing until the required data block is located.

Advantages of Multi-Level Indexing

- Reduces the number of index entries examined during a search.
- Makes searching efficient even when the data file is extremely large.
- Reduces disk I/O compared with searching a very large single-level index.
- Provides better scalability as the database grows.

Conclusion

Single-level indexing is simpler and works well for smaller databases, but its index can become large and expensive to search. **Multi-level indexing** adds additional index levels to organize the index itself, resulting in **faster and more scalable retrieval for large databases**.

In short:

Single-level → simple, suitable for smaller files.

Multi-level → hierarchical, faster and more scalable for large files.

10. Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.

1. Hashing vs. Indexing for Database Queries

Hashing and indexing are both used to speed up database retrieval, but their performance differs significantly depending on whether the query is an **exact-match** query or a **range** query.

Query / Factor	Hashing	Indexing (B/B+ Tree)
Exact-match query	**Excellent** — typically near O(1) average access with a good hash function.	Very good — typically O(log N) for a B/B+ Tree.
Range query	**Poor** — hash values do not preserve the ordering of keys.	**Excellent** — especially B+ Trees, because keys are ordered and leaf nodes are linked.
Example exact match	WHERE Student_ID = 105	WHERE Student_ID = 105
Example range	WHERE Salary BETWEEN 30000 AND 50000 → inefficient	WHERE Salary BETWEEN 30000 AND 50000 → efficient
Ordering	Does not maintain key order.	Maintains keys in sorted order.
Sequential access	Not efficient.	Efficient.
Collision/Overflow	Possible; can reduce performance.	No hashing collisions; tree may require node splits.
Insertion/Deletion	Usually fast, particularly with dynamic hashing.	Can require tree restructuring and index maintenance.
Best application	Exact-key lookups.	Exact lookups **and especially range/ordered queries**.

1. Exact-Match Queries

An exact-match query looks for a particular value:

```
SELECT * FROM Employee
WHERE Employee_ID = 1050;
```

Hashing is usually the better choice because the hash function maps 1050 directly to the bucket containing the record. With a well-designed hash table and low collision rate, average lookup is approximately **O(1)**.

An indexed structure such as a **B+ Tree** also performs very well, but lookup generally takes **O(log N)** because the tree must be traversed from the root to a leaf.

Evaluation: For frequent exact-match searches, **hashing generally has the performance advantage**.

2. Range Queries

A range query retrieves multiple values within an interval:

SELECT * FROM Employee

WHERE Salary BETWEEN 30000 AND 50000;

Hashing performs poorly here because values such as 30000, 40000, and 50000 can be distributed into completely unrelated buckets. The database may therefore need to examine many or all buckets.

A **B+ Tree index**, however, keeps keys ordered. The database can find the first matching value and then follow the linked leaf nodes to retrieve the remaining values efficiently.

Evaluation: For range queries, **B+ Tree indexing is clearly superior to hashing**.

Overall Evaluation

- **Exact-match queries: Hashing → best performance**, assuming a good hash function and controlled collisions.
- **Range queries: B+ Tree indexing → best performance** because it preserves key ordering.
- **Mixed workloads: B+ Tree indexing** is often more versatile because it supports both exact-match and range queries efficiently.
- **Large, frequently changing datasets:** Dynamic hashing can provide efficient exact-match access, while B+ Trees offer strong performance for ordered and range-based access.

Conclusion

There is no single best technique for every workload:

Hashing → optimized for "Find this exact record."

Indexing (especially B+ Trees) → optimized for "Find records in this range/order."

Therefore, the choice should be based primarily on the **query pattern** of the database application.